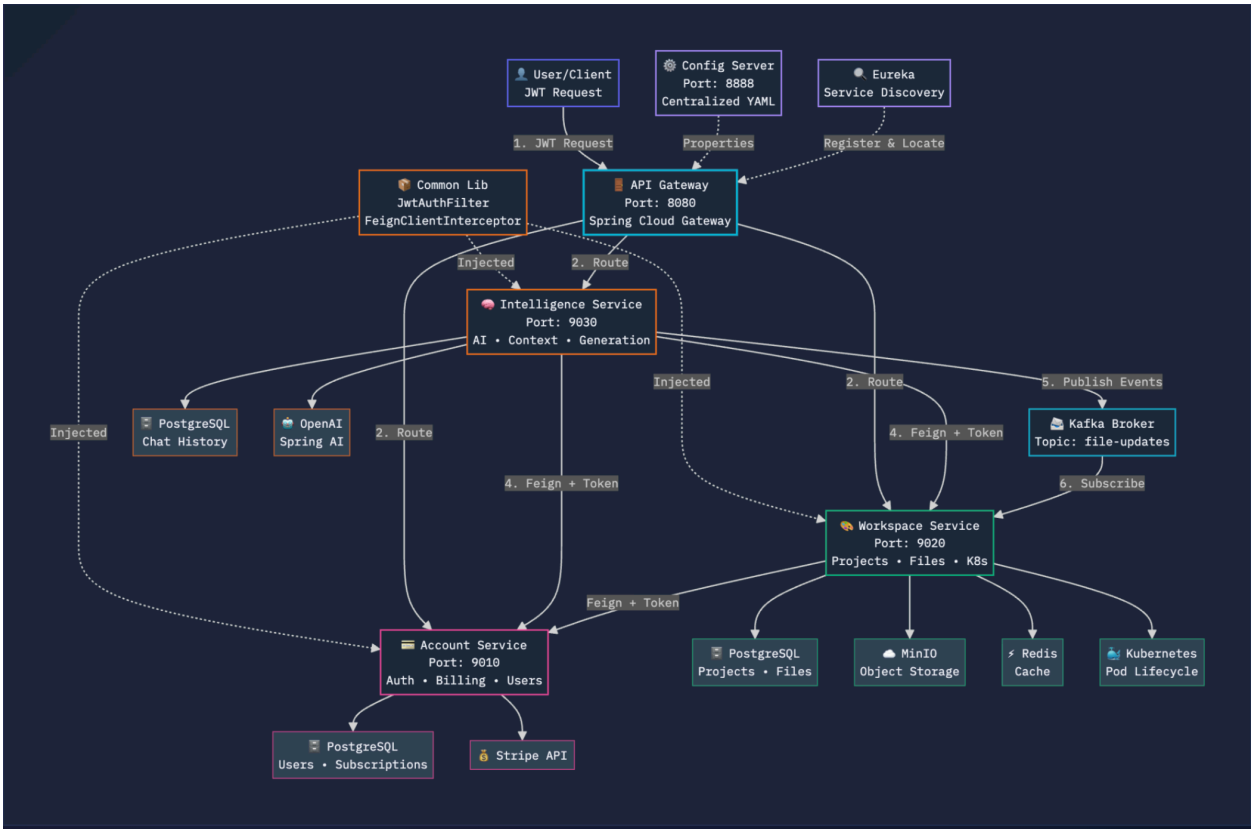
Distributed Lovable Architecture

Microservices

Microservice	Primary Responsibility	Database / Storage	Key Integrations	Cross-Service Calls (Feign)
Account Service	User Identity, Authentication, and Stripe Billing.	PostgreSQL (Users, Plans, Subscriptions)	Stripe API	*None*
Workspace Service	Project management, File Tree, Kubernetes Deployments.	PostgreSQL (Projects, Members), **MinIO** (Files)	Kubernetes API, Redis	Calls `AccountService` (Check limits, users)
Intelligence Service	AI Chat, Context gathering, Code Generation.	PostgreSQL (Chat Sessions, Messages, Events)	OpenAI (Spring AI)	Calls `WorkspaceService` (Read files), `AccountService` (Check AI limits)



Account Service

Category	Components & Details
Dependencies	`spring-boot-starter-web`, `spring-data-jpa`, `postgresql`, `stripe-java`, `mapstruct`, `spring-cloud-starter-netflix-eureka-client`
Key Entities	`User`, `Plan` (Free, Pro), `Subscription`
Security Setup	`AccountSecurityConfig`: Exposes `/auth/**` and `/webhooks/stripe` publicly. Secures everything else.
Internal API	`InternalAccountController`: Exposes `/internal/v1/...` for other microservices to fetch user DTOs and verify billing plans.

Workspace Service

Category	Components & Details
Dependencies	`spring-boot-starter-web`, `spring-data-jpa`, `minio`, `kubernetes-client` (Fabric8), `spring-data-redis`, `spring-cloud-starter-openfeign`
Key Entities	`Project`, `ProjectMember`, `ProjectFile`, `Preview` (K8s pod mapping)

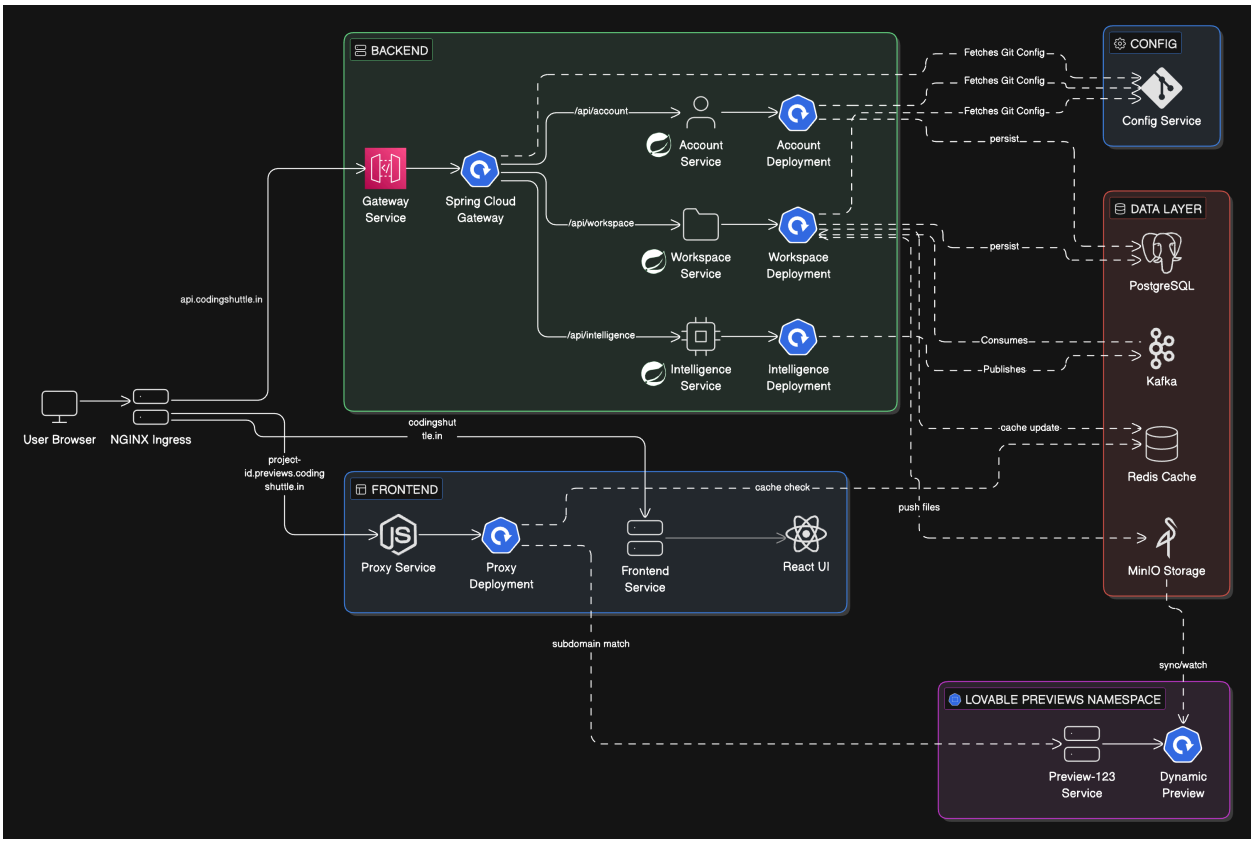
Cloud Configs	`StorageConfig` (MinIO connection), `KubernetesConfig`, `RedisConfig`
Security Logic	`SecurityExpressions`: Contains custom `@PreAuthorize("@security.canEditProject(#id)")` logic to verify ownership via `ProjectMemberRepository`.
Feign Clients	`AccountClient`: Used during Project Creation to verify if the user has hit their `maxProjects` limit.

Intelligence Service

Category	Components & Details
Dependencies	`spring-ai-starter-model-openai`, `spring-data-jpa`, `spring-cloud-starter-openfeign`
Key Entities	`ChatSession`, `ChatMessage`, `ChatEvent` , `UsageLog` (Tracks LLM thoughts, file edits, tool uses)
AI Components	`AiGenerationServiceImpl` (Prompt execution), `LLMResponseParser` (Regex to extract `<file>` and `<tool>` tags).
Advisors & Tools	`FileTreeContextAdvisor` (Injects current file tree into prompt), `CodeGenerationHelperTools` (Allows LLM to read specific files).
Feign Clients	`WorkspaceClient` (To fetch file trees and file content for context), `AccountClient` (To verify AI daily token limits).

Common Lib

Component in common-lib	What it does	Why it's needed
`JwtAuthFilter`	Extracts the Bearer token and populates the `SecurityContext`.	Every service authenticates requests locally, completely independent of the Gateway.
`FeignClientInterceptor`	Automatically grabs the user's JWT from the context and attaches it to outbound Feign requests.	Allows the Intelligence service to call the Workspace service seamlessly as the logged-in user.
Shared DTOs	`UserDto`, `PlanDto`, `FileTreeDto`, etc.	Prevents JPA `@Entity` classes from leaking across microservice boundaries.



Step 1: Connect to the GKE Cluster

"First, we need to bridge our local terminal to our Google Cloud infrastructure. This command downloads the authentication certificates so our local `kubectl` *commands are securely routed to our GKE cluster."*

```
gcloud container clusters get-credentials lovable-me-cluster --region asia-south1 --project project-9ca46dba-039d-4038-bfe
```

Verify the connection for the class:

Step 2: Create the Namespaces

"Kubernetes namespaces are like virtual clusters inside our main cluster. We are creating `lovable-core` *for our main application, and* `lovable-previews` *to isolate our dynamic user previews."*

```
kubectl create namespace lovable-core
kubectl create namespace lovable-previews
```

Step 3: Apply Base Configurations (Secrets & ConfigMaps)

"Before we boot up any applications, we must load their environment variables and secure credentials (like our GitHub token and database passwords) into the cluster's memory."

(Note: Adjust the folder paths below if your files are organized differently).

Bash

```
kubectl create secret generic app-secrets --from-env-file=.env -n lovable-core
kubectl apply -f k8s/configmaps/ -n lovable-core
```

Step 4: Deploy Stateful Infrastructure (Databases & Brokers)

"Always deploy your stateful infrastructure first. Our Spring Boot services will crash on startup if PostgreSQL, Redis, and Kafka aren't already running and accepting connections."

```
kubectl apply -f k8s/databases/ -n lovable-core
```

(Press `Ctrl+C` to exit the watch mode once they are running).

Step 5: Deploy the Microservices (Deployments & Services)

"Now that our databases are ready, we will apply the Deployments (which pull our Jib-built Docker images and run the pods) and the Services (which give each pod an internal DNS name so they can talk to each other)."

Bash

```
kubectl apply -f k8s/services/ -n lovable-core
```

Step 6: Install the NGINX Ingress Controller

"Right now, our app is running, but it is completely isolated from the public internet. We need an Ingress Controller to act as our front door. We are installing NGINX, which will automatically provision a Google Cloud Network Load Balancer for us."

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.8.2/deploy/static/provider/cloud/deploy.yaml
```

Crucial Step: You must wait for Google to assign a public IP address. Run this command with the watch flag (`-w`):

```
kubectl get svc ingress-nginx-controller -n ingress-nginx -w
```

Speaker Note: *"See where it says `<pending>`? Google Cloud is currently allocating a public IPv4 address for our load balancer. Ah, there it is!"(Copy the `EXTERNAL-IP` that appears, you will need it for the DNS/GoDaddy step).*

Apply the Ingress Routing Rules

"Now that NGINX is installed and listening on a public IP, we need to give it our routing map. This YAML tells NGINX to send `/api` traffic to the Spring Cloud Gateway, and everything else to our Node.js Proxy."

```
kubectl apply -f k8s/ingress.yaml -n lovable-core
```